Software Security Assignment I

Booking Management System

Field	**Details**
Author	Bassam Salou
Course	Software Security
Date	October 2026
Repository	https://github.com/bassamsalou/software-security-assignment.git



Indhold

1 Introduction and Project Scope 3
2 Architecture, Trust Boundaries and Authority 3
3 Security Lifecycle, Testing and CI/CD 5
4 Vulnerability Investigation 7
4.1 V1 – CWE-862: Missing Authorization 7
4.2 V2 – CWE-89: SQL Injection 8
4.3 V3 – CWE-20: Improper Input Validation 9
4.4 V4 – Stale Authorization Between Services 10
5 Secure Design Principles 11
5.1 Least Privilege 11
5.2 Complete Mediation 11
5.3 Fail-Safe Defaults 11
6 Three-Component Workflow and Authority Analysis 12
7 Release, Deployment and Production Considerations 13
8 Evidence and Traceability 13
9 Limitations 15
10 Conclusion 15

1 Introduction and Project Scope

The purpose of this project was to develop a small multi-tier Booking Management System while applying security throughout the development lifecycle. The application uses synthetic data and allows authenticated users to access their own bookings, while an Administrator has additional management permissions.

The system was implemented in Python and consists of three separate components: an API Service, a Resource Service, and a Processing Service. They communicate through HTTP, and the Processing Service uses a separately installed Python helper package for generating booking summaries.

Before implementation, I created a Security Lifecycle Plan covering requirements, assets, trust boundaries, access control, threats, testing, and release gates. Three secure design principles were selected: least privilege, complete mediation, and fail-safe defaults.

Four vulnerabilities were intentionally investigated. Three came from the course catalogue: CWE-862 Missing Authorization, CWE-89 SQL Injection, and CWE-20 Improper Input Validation. The fourth was a composition vulnerability involving stale authorization between services. Each vulnerability was demonstrated and then fixed, with the vulnerable and fixed versions preserved in separate Git commits.

The final application is supported by automated functional, authorization, security, and integration tests. Jenkins is used to run the tests in a fresh environment, perform static analysis with Bandit, audit dependencies with pip-audit, and save the results as build evidence.

2 Architecture, Trust Boundaries and Authority

The application consists of three independently running services with separate responsibilities. The API Service runs on port 5000 and is the main entry point. It authenticates the seeded users and creates a session token linked to the user's username and role. This means the API gets the user's authority from the authenticated session instead of allowing the client to choose a username or role.

The Resource Service runs on port 5001 and owns the bookings, ownership information, and generated summaries. It is the main authorization boundary. Ordinary users can only access resources they own, while the Administrator has additional permissions. If the required identity or authority cannot be verified, the request is denied.

The Processing Service runs on port 5002 and handles booking-summary generation. It does not have direct database access. Instead, it requests the required booking from the Resource Service and uses the separately installed booking-helper package to generate the summary. The Resource Service is also responsible for storing the finished summary.

Several trust boundaries exist in the system. User-controlled data crosses the boundary between the user and API Service, while identity, role, booking data, and processing requests cross boundaries

between the three services. There is also a boundary between the Processing Service and booking-helper. Data is therefore not trusted only because it came from another component.

The API Service establishes the authenticated identity, but the Resource Service makes the final authorization decision for protected resources. During the summary workflow, it checks authorization when a booking is retrieved and checks the current ownership again before the summary is stored. This is especially important for preventing the stale-authorization vulnerability discussed later.

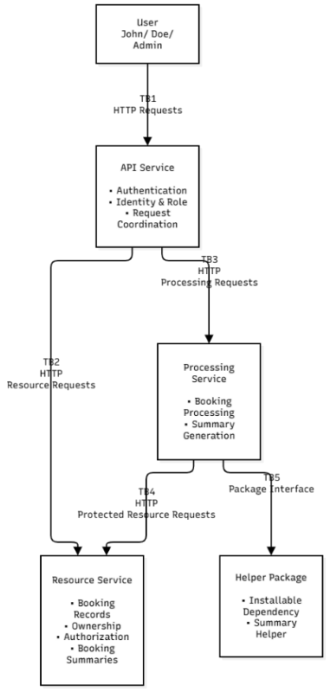


Figure 1 - System Architecture and Trust Boundaries.

3 Security Lifecycle, Testing and CI/CD

Security was included from the beginning of the project. Before implementing the main functionality, I created a Security Lifecycle Plan covering requirements, protected assets, architecture, trust boundaries, access control, threats, testing, and release gates. An asset register identified booking records, ownership information, summaries, authentication information, and user roles as important assets.

I also used STRIDE to identify possible threats. Information disclosure was relevant because users could try to access other users' bookings, elevation of privilege was relevant to administrator functionality, and tampering was relevant to booking data and information sent between services. These threats helped identify where authorization and input validation were needed.

Automated testing was used to check the security requirements. The final project has 29 tests: 21 covering the API Service, Resource Service, authentication, authorization, and security behaviour, and eight integration tests. The integration tests automatically start all three services and test communication between them, including the booking-summary workflow.

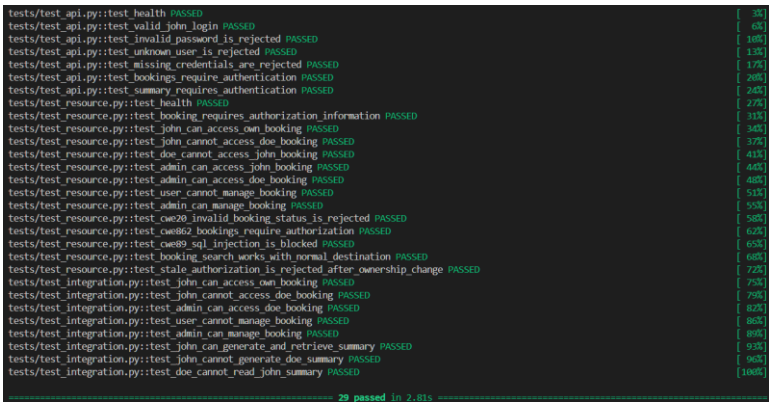


Figure 2 - Final automated test suite with 29 passing tests.

Jenkins was used for CI. The pipeline creates a fresh Python environment, installs the dependencies and the separate booking-helper package, validates the code, and runs the automated tests. This also checks that the application can be reproduced without depending on packages already installed on my computer.

Bandit and pip-audit were added as security gates. Bandit reports zero Medium and zero High findings. It reports three Low findings caused by the hardcoded passwords for the synthetic test

accounts. These are accepted for the local assignment environment, while Medium or High findings block the build.

pip-audit checks the installed dependencies for known vulnerabilities. During development, it detected vulnerable setuptools versions and caused the pipeline to fail. The Jenkins environment was updated to upgrade setuptools, after which the audit reported no known vulnerabilities.

The final pipeline runs all 29 tests and archives the JUnit test results, Bandit report, and pip-audit report as evidence. A failed test, Medium or High Bandit finding, or known vulnerable dependency causes the pipeline to fail.

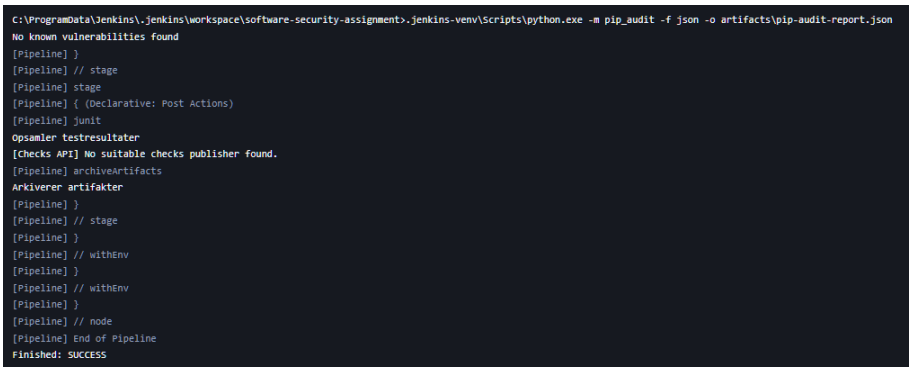


Figure 3 - Successful Jenkins pipeline after automated testing and security analysis.

4 Vulnerability Investigation

Four vulnerabilities were intentionally introduced and tested during the project. Three of them were selected from the course vulnerability catalogue, while the fourth was a composition vulnerability involving authorization across multiple services. Each vulnerability was first implemented in a vulnerable state and committed to Git. After demonstrating the problem, a fix was implemented in a separate commit together with tests for the expected secure behaviour.

4.1 V1 – CWE-862: Missing Authorization

The first vulnerability was CWE-862, Missing Authorization. In the vulnerable version, the Resource Service could return a booking based on its ID without checking whether the authenticated user owned it. This meant that John could request a booking belonging to Doe even though he should only have access to his own bookings. The vulnerable version is stored in commit cb49f21.

The fix was to enforce authorization in the Resource Service. Ordinary users can now only access bookings where the stored owner matches their authenticated username, while the Administrator can access any booking. The fix is stored in commit 13ea2e5. Automated tests verify that John can access his own booking but cannot access Doe's booking.

This showed that authentication alone is not enough. Authorization must also be checked for the specific resource being accessed.

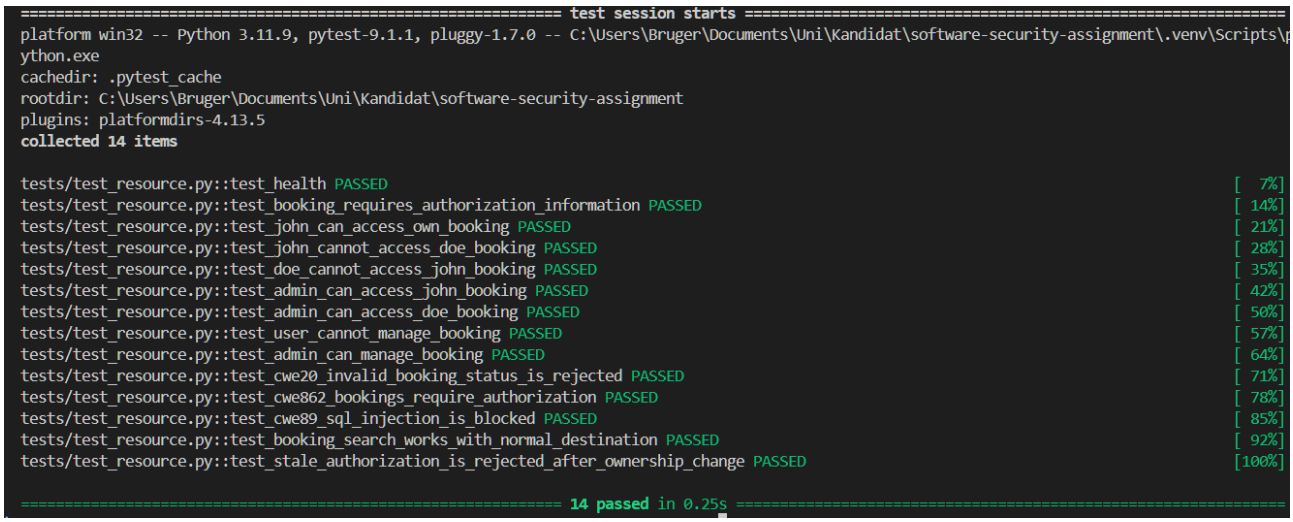


Figure 4 - Automated authorization tests showing that users are restricted to their own bookings.

4.2 V2 – CWE-89: SQL Injection

The second vulnerability was CWE-89, SQL Injection, and was introduced in the booking search functionality. In the vulnerable version, the destination supplied by the user was inserted directly into the SQL query. A specially constructed input could therefore change how the database interpreted the query. This version is stored in commit 12eb220.

The vulnerability was fixed by using parameterized SQL queries. The destination and username are now supplied separately as parameters instead of becoming part of the SQL statement. Authorization is still applied to the search, meaning ordinary users remain restricted to their own bookings. The fix is stored in commit facbecd.

A regression test verifies that the previous SQL injection input can no longer change the query behaviour. This demonstrated why external input should be treated as data instead of being inserted directly into SQL statements.

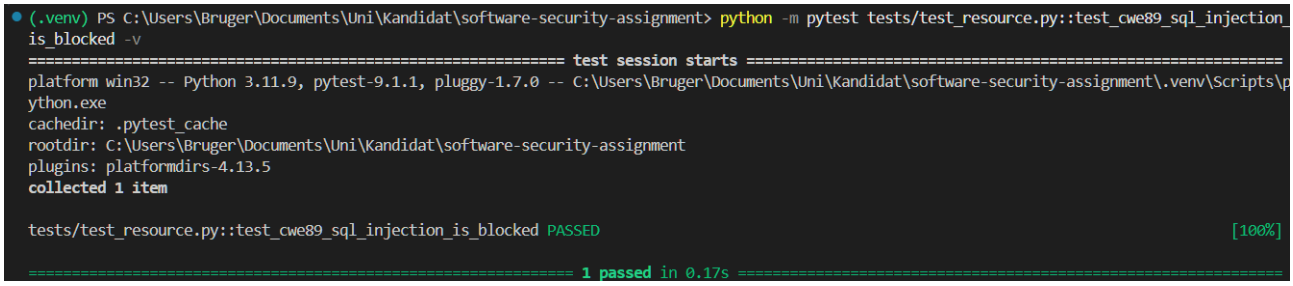


Figure 5 - Automated regression test showing that the SQL injection attempt is rejected by the fixed implementation.

4.3 V3 – CWE-20: Improper Input Validation

The third vulnerability was CWE-20, Improper Input Validation. It was introduced in the administrator operation for changing a booking's status. The vulnerable implementation accepted status values without checking whether they were valid, allowing unexpected values to be stored in the database. This version is stored in commit 04c3785.

The fix was to add an allowlist containing the valid values pending, confirmed, and cancelled. Missing or unsupported status values are now rejected. The fix is stored in commit ca4556d, and a regression test verifies that an invalid status cannot be stored.

This showed that authorization and input validation have different purposes. An Administrator may have permission to update a booking while still providing invalid data, so both checks are required.

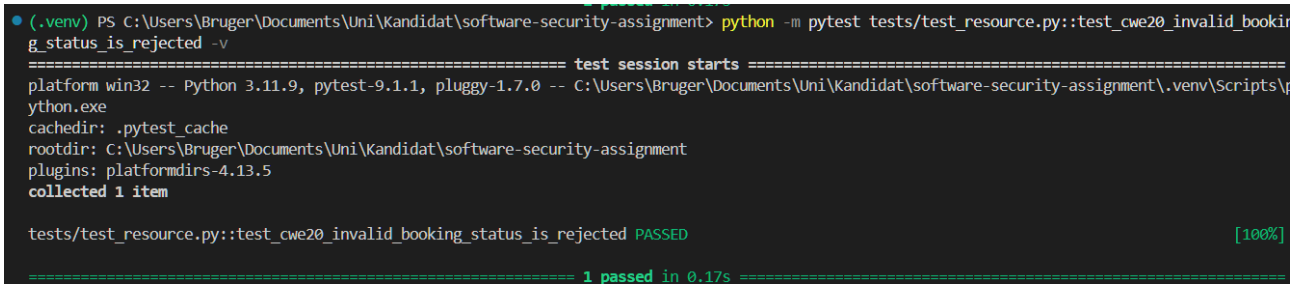


Figure 6 - Automated regression test showing that an invalid booking status is rejected.

4.4 V4 – Stale Authorization Between Services

The fourth vulnerability was a composition vulnerability involving stale authorization. John could initially be authorized to access a booking and start summary generation. However, the booking owner could change to Doe before the generated summary was stored.

In the vulnerable version, the Resource Service trusted the ownership information obtained earlier in the workflow. John could therefore complete the operation using stale authorization even though he was no longer the booking owner. The vulnerable version is stored in commit 054e0d0.

The fix was to re-check authorization when the summary is stored. The Resource Service now re-fetches the booking, checks its current owner, and verifies that it matches the owner supplied with the summary. If ownership has changed, the request is rejected. The fix is stored in commit 6ff9d1a.

A regression test reproduces the ownership change and verifies that the stale request is rejected. This demonstrates complete mediation because authorization is checked again when the protected operation actually takes place instead of relying on an earlier decision.

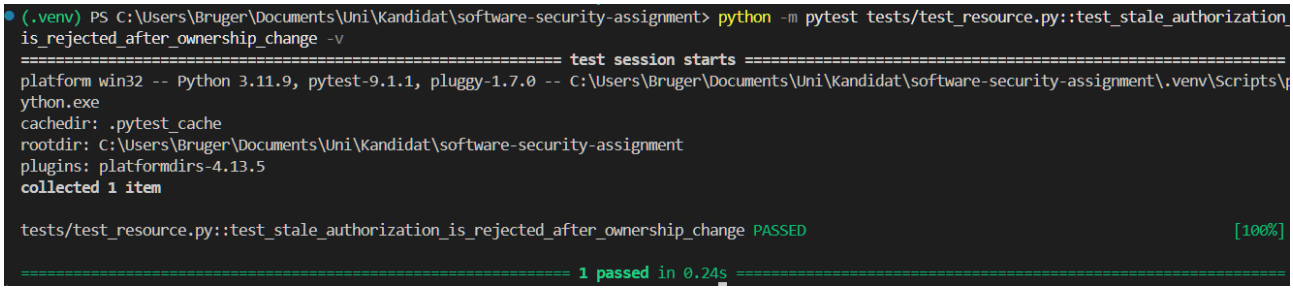


Figure 7 - Regression test showing that stale authorization is rejected after booking ownership changes.

5 Secure Design Principles

Three secure design principles were selected at the beginning of the project: least privilege, complete mediation, and fail-safe defaults. Instead of only describing these principles in the Security Lifecycle Plan, I tried to apply them directly in the implementation and security tests.

5.1 Least Privilege

Least privilege is applied through the separation of roles and service responsibilities. Ordinary users can only access their own bookings and summaries and cannot use administrator operations. The Processing Service also has no direct access to the booking database. Instead, it requests the required data from the Resource Service, where authorization is checked.

This limits users and components to the access they actually need.

5.2 Complete Mediation

Complete mediation is applied by checking authorization when a protected resource is accessed. The Resource Service checks the current booking owner instead of relying only on an earlier authorization decision.

This was especially important for the stale-authorization vulnerability in Section 4.4. Before storing a generated summary, the Resource Service re-fetches the booking and checks its current ownership. This prevents an earlier valid authorization from being reused after ownership has changed.

5.3 Fail-Safe Defaults

Fail-safe defaults are used by rejecting requests when the required authorization information is missing or invalid. Ordinary users are not given administrator permissions, unknown roles are denied, and users cannot access a booking unless their authority can be verified.

The same approach is used for input validation. Booking statuses outside pending, confirmed, and cancelled are rejected instead of being accepted by default.

Together, these principles limit unnecessary access, ensure authorization is checked at the protected-resource boundary, and make invalid or unauthorized requests fail safely.

6 Three-Component Workflow and Authority Analysis

The booking-summary functionality is the operation that uses all three application services. The user first sends the request to the API Service, which uses the session token to identify the username and role. The API then sends the booking ID and authenticated authority to the Processing Service.

The Processing Service requests the booking from the Resource Service because it does not have direct database access. The Resource Service checks whether the user is allowed to access the booking. If authorized, the booking is returned and the Processing Service uses booking-helper to generate the summary.

The summary is then sent to the Resource Service for storage. Before storing it, the Resource Service retrieves the current booking again and checks its ownership. This prevents stale authorization from being used if ownership changed during the workflow.

The API Service therefore establishes the user's identity, the Processing Service performs the processing task, and the Resource Service makes the final authorization decisions over protected data.

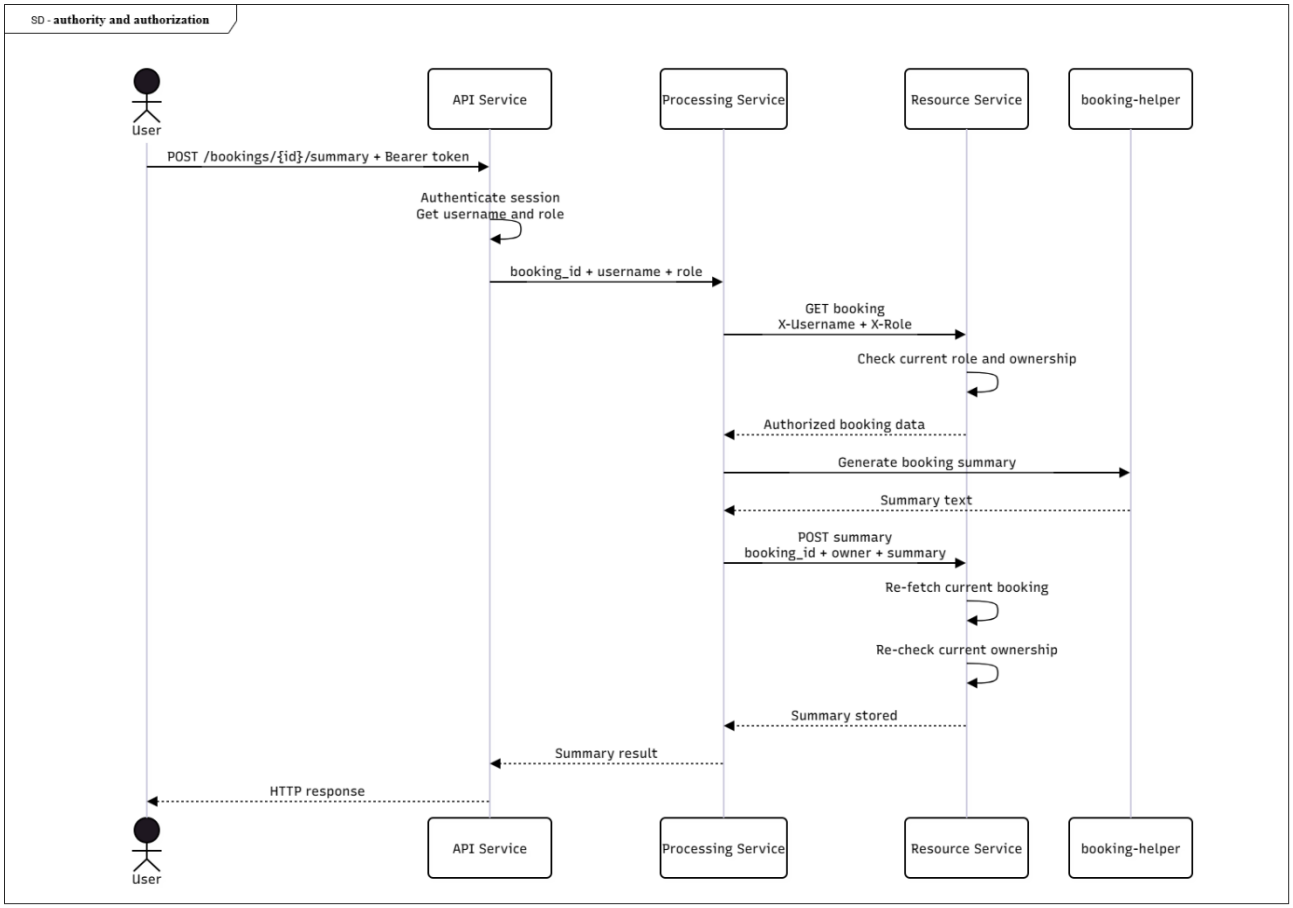


Figure 8 - Booking-summary sequence showing the flow of authority and authorization checks across all three application services.

7 Release, Deployment and Production Considerations

The application is designed for a local assignment environment using synthetic users and data. Some parts would therefore need stronger security before a real production deployment.

The seeded passwords are stored directly in the application for reproducible testing, which causes the three Low Bandit findings. In production, passwords should instead be securely hashed and secrets should not be stored in the source code.

Communication would also need stronger protection. The current services communicate over HTTP and pass username and role information between them. A production system should use HTTPS/TLS and service-to-service authentication so an attacker cannot simply provide false identity information to an internal service.

Production deployment would also require secure secret management, security logging and monitoring. SQLite is sufficient for this project, but a real system would need stronger database access controls, backup procedures and deployment security.

The CI security gates would still be useful for release decisions. Failed tests, Medium or High Bandit findings, or known vulnerable dependencies should prevent a version from being released until the problem has been investigated.

8 Evidence and Traceability

Git was used throughout the project to keep the different stages of development separate and traceable. The Security Lifecycle Plan was committed before the main application functionality was implemented. The intentionally vulnerable versions were also committed before their fixes, which makes it possible to compare the vulnerable and secure implementations.

The table below lists some of the main commits used as evidence in the project.

Evidence	Git commit	Description
Initial project structure	75ee3d0	Created the initial project structure
Initial requirements	007f70c	Added the initial application requirements
Initial lifecycle plan	d32f66c	Added the initial Security Lifecycle Plan
Completed initial lifecycle plan	1b1a76e	Completed the Security Lifecycle Plan before feature implementation
Authentication	ed05c54	Implemented API authentication and booking access
Authorization	f3775be	Added authorization for specific booking access
Three-component workflow	be352ae	Implemented the Processing Service and booking-summary workflow
Automated tests	51efa08	Added automated functional and authorization tests
V1 – CWE-862 vulnerable	cb49f21	Demonstrated missing authorization
V1 – CWE-862 fixed	13ea2e5	Enforced authorization in the Resource Service
V2 – CWE-89 vulnerable	12eb220	Demonstrated SQL injection
V2 – CWE-89 fixed	facbecd	Replaced vulnerable SQL with parameterized queries
V3 – CWE-20 vulnerable	04c3785	Demonstrated improper input validation
V3 – CWE-20 fixed	ca4556d	Added an allowlist for booking status values
V4 – Stale authorization vulnerable	054e0d0	Demonstrated the composition vulnerability
V4 – Stale authorization fixed	6ff9d1a	Added re-authorization using complete mediation
Security analysis evidence	8060f36	Added security and dependency analysis evidence
Initial Jenkins pipeline	2c9e8e8	Added the initial CI pipeline
Automated Jenkins tests	5231e84	Added automated tests to Jenkins
Jenkins integration tests	9a8f2ae	Added integration tests to the pipeline
Bandit security gate	1086224	Added the Bandit security gate
Dependency security gate	f8f8b6d	Added dependency auditing to Jenkins
Dependency fix	bca925f	Upgraded setuptools in the Jenkins environment
Jenkins evidence	5726543	Archived Jenkins security and test evidence
Reproducible setup	6da531b	Added project setup and security documentation to README

Together with the automated tests and archived Jenkins artifacts, the Git history shows the development from the initial security planning to implementation, vulnerability testing, security fixes and automated security checks. In particular, keeping the vulnerable and fixed versions in separate commits makes the security changes easier to identify and reproduce.

9 Limitations

The project was intentionally kept small and runs locally using synthetic data and three predefined users. Features such as registration, a GUI, payments, email, public hosting, high availability, and large-scale deployment were outside the project scope.

The communication between services is simplified, and a production system would require stronger service-to-service authentication. SQLite and the hardcoded seeded accounts are also suitable only for the local assignment environment.

The booking-helper package is developed locally and installed separately. Because it is not published on PyPI, pip-audit cannot check it against PyPI vulnerability information like the external dependencies.

The optional supply-chain experiment was not performed. The project instead focused on the required architecture, security lifecycle, three catalogue vulnerabilities, and the additional composition vulnerability.

10 Conclusion

The project resulted in a small multi-tier Booking Management System consisting of three Python services and a separately installed helper package. Security was considered from the planning stage through implementation, testing, and CI.

The four vulnerability investigations showed different security problems in practice. CWE-862 demonstrated the importance of authorization, CWE-89 showed the need for parameterized SQL, CWE-20 showed the importance of validating input, and the stale-authorization vulnerability showed why authorization may need to be checked again during a multi-service workflow.

The final application has 29 automated tests, while Jenkins performs testing, static analysis with Bandit, and dependency auditing with pip-audit. The dependency gate also detected a vulnerable dependency during development, showing that the automated security checks could actually stop an unsafe build.

Overall, the project showed the value of considering security throughout development rather than only testing for security problems at the end. The combination of security planning, secure design principles, vulnerability testing, regression tests, and CI security gates helped identify and fix problems in a structured way.

Appendix

1. AI Declaration

Generative AI (ChatGPT) was used as a supporting tool during this assignment. It was mainly used for guidance during implementation and debugging, suggestions for automated tests and Jenkins configuration, explanations of security concepts, and help with structuring and improving the wording of the report.

AI-generated suggestions were reviewed and adapted before being used. The application was implemented and tested in my own development environment, and the security vulnerabilities, fixes, automated tests, Git history, and Jenkins pipeline were verified through the project repository and local/Jenkins test results.

2. CI/CD Evidence

Automatic SCM trigger

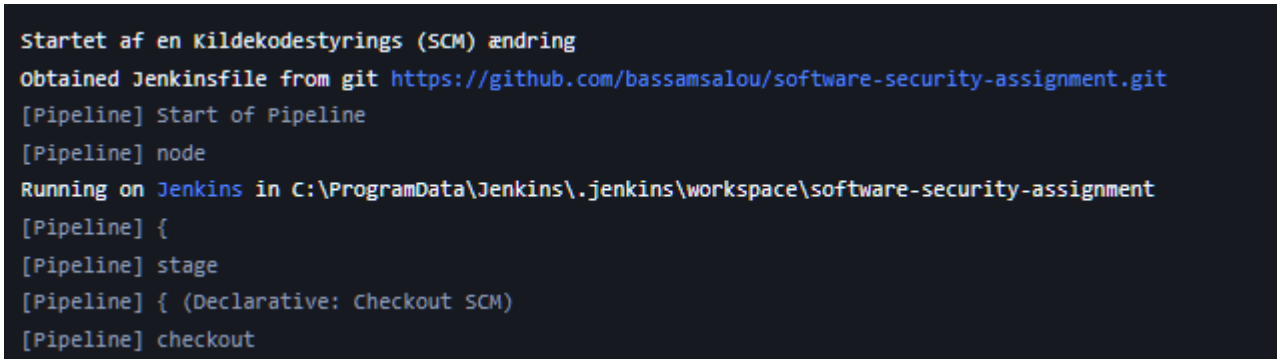


Figure 9 - screenshot showing Jenkins started the build automatically after your Git push.

Successful pipeline



Figure 10 - screenshot showing the same build ending with Finished: SUCCESS.